CAPÍTULO

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 **17** 18 19 20 21 22 23 24 25 26

Engenharia de software baseada em componentes

Objetivos

O objetivo deste capítulo é descrever uma abordagem sobre o reúso de software baseado na composição de componentes reusáveis, padronizados. Com a leitura deste capítulo, você:

- saberá que a engenharia de software baseada em componentes está preocupada com o desenvolvimento dos componentes padronizados baseados em modelos de componentes, além da composição destes em sistemas de aplicações;
- compreenderá o que se entende por um componente e um modelo de componente;
- conhecerá as principais atividades do processo CBSE para reúso e o processo CBSE com reúso;
- entenderá algumas das dificuldades e problemas que possam surgir durante o processo de composição de componentes.

Conteúdo

17.1 Componentes e modelos de componentes
17.2 Processos CBSE
17.3 Composição de componentes

Como explicado no Capítulo 16, atualmente muitos novos sistemas de negócios são desenvolvidos pela configuração de sistemas disponíveis no mercado. No entanto, quando uma empresa não pode usar um sistema de prateleira, porque eles não atendem a seus requisitos, o software de que necessitam precisa ser especialmente desenvolvido. Para o desenvolvimento de softwares customizados, a engenharia de software baseada em componentes é uma forma eficaz, orientada ao reúso, de desenvolver novos sistemas corporativos.

A engenharia de software baseada em componentes (CBSE, do inglês *component-based software engineering*) surgiu na década de 1990 como uma abordagem para softwares de desenvolvimento de sistemas com base no reúso de componentes de softwares. Sua criação foi motivada pela frustração de projetistas, pois o desenvolvimento orientado a objetos não levou a um amplo reúso, como se havia sugerido. As classes de objetos foram muito detalhadas e específicas e muitas vezes precisavam ser associadas com uma aplicação em tempo de compilação. Era preciso ter conhecimento detalhado das classes para usá-las e isso geralmente significava que era necessário ter o código-fonte do componente, o que significava que vender ou distribuir objetos como componentes reusáveis individuais era praticamente impossível.

Os componentes são abstrações de nível mais alto do que objetos e são definidos por suas interfaces. Geralmente, eles são maiores que objetos individuais e todos os detalhes de implementação são escondidos de outros componentes. CBSE é o processo de definir, implementar, integrar ou compor componentes independentes, pouco acoplados em sistemas. Essa se tornou uma importante abordagem de desenvolvimento de software porque os sistemas de software estão ficando maiores e mais complexos. Os clientes exigem softwares mais confiáveis, entregues e implantados mais rapidamente.

A única maneira pela qual podemos lidar com a complexidade e entregar o melhor software, mais rapidamente, é reusar em vez de reimplementar os componentes de software.

Os fundamentos da engenharia de software baseada em componentes são:

1. Os componentes independentes que são completamente especificados por suas interfaces. Deve haver uma separação clara entre a interface de componente e sua implementação. Isso significa que a implementação de um componente pode ser substituída por outra, sem que se alterem outras partes do sistema.
2. Os padrões de componentes que facilitam a integração destes. Essas normas são incorporadas a um modelo de componentes. Eles definem, no mínimo, como interfaces de componentes devem ser especificadas e como os componentes se comunicam. Alguns modelos vão muito mais longe e definem as interfaces que devem ser implementadas por todos os componentes. Se os componentes estão em conformidade com os padrões, sua operação é independente de sua linguagem de programação. Componentes escritos em linguagens diferentes podem ser integrados ao mesmo sistema.
3. O *middleware* que fornece suporte de software para a integração de componentes. Para tornar independentes, os componentes distribuídos trabalham juntos; você precisa de suporte de *middleware* que lide com as comunicações de componentes. O *middleware* para suporte ao componente lida, com eficiência, com questões de nível inferior e permite que você se concentre nos problemas relacionados com a aplicação. Além disso, o *middleware* para suporte de componentes pode fornecer suporte para alocação de recursos, gerenciamento de transações, proteção e concorrência.
4. Um processo de desenvolvimento que é voltado para a engenharia de software baseada em componentes. Você precisa de um processo de desenvolvimento que permita que os requisitos evoluam, dependendo da funcionalidade dos componentes disponíveis. Na Seção 17.2, eu discuto os processos de desenvolvimento de CBSE.

O desenvolvimento baseado em componentes encarna as boas práticas da engenharia de software. Faz sentido projetar um sistema usando componentes, mesmo que seja necessário desenvolver, em vez de reusar esses componentes. A CBSE de base apoia-se em um dos princípios de projeto na construção de softwares compreensíveis e passíveis de manutenção:

1. Componentes são independentes, então eles não interferem na operação uns dos outros. Detalhes de implementação são ocultados. Implementação dos componentes pode ser alterada sem afetar o restante do sistema.
2. Os componentes comunicam-se por meio de interfaces bem definidas. Se essas interfaces forem mantidas, um componente poderá ser substituído por outro, que forneça funcionalidade adicional ou aumentada.
3. As infraestruturas dos componentes oferecem uma gama de serviços-padrão que podem ser usados em sistemas de aplicações, o que reduz a quantidade de códigos novos a serem desenvolvidos.

A motivação inicial para CBSE foi a necessidade de apoiar o reúso e a engenharia de software distribuída. Um componente era percebido como um elemento de um sistema de software que podia ser acessado pelo uso de um mecanismo de chamada de procedimento remoto, por outros componentes em execução em computadores separados. Cada sistema que reusou um componente precisava incorporar a própria cópia desse componente. Essa ideia de um componente estendia a noção de objetos distribuídos, conforme definido em modelos de sistemas distribuídos, como a especificação CORBA (POPE, 1997). Uma variedade de protocolos e normas foi desenvolvida para dar suporte a essa visão de um componente, como a Enterprise Java Beans (EJB) da Sun, COM e .NET da Microsoft, bem como CCM da CORBA (LAU e WANG, 2007).

Na prática, esses vários padrões têm dificultado a absorção de CBSE. Era impossível para os componentes desenvolvidos trabalharem juntos usando abordagens diferentes. Componentes que são desenvolvidos para diferentes plataformas, como .NET ou J2EE, não podem interoperar. Além disso, as normas e os protocolos propostos eram complexos e difíceis de serem compreendidos. Esse também foi um obstáculo a sua adoção.

Em resposta a esses problemas, a noção de um componente como um serviço foi desenvolvida e normas foram propostas para apoiar a engenharia de software orientada a serviços. A diferença mais significativa entre um componente como um serviço e a noção original de um componente é que os serviços são entidades autônomas externas ao programa que os usa. Quando você cria um sistema orientado a serviços, é melhor referenciar o serviço externo do que incluir uma cópia desse serviço em seu sistema.

A engenharia de software orientada a serviços, que discuto no Capítulo 19, é, portanto, um tipo de engenharia de software baseada em componentes. Ela usa uma noção mais simples de um componente do que o inicialmente proposto pela CBSE. Ela tem sido impulsionada, desde o início, por normas. Em situações nas quais o reúso baseado em um COTS é impraticável, a CBSE orientada a serviços está se tornando a abordagem dominante para o desenvolvimento de sistemas de negócios.

17.1 Componentes e modelos de componentes

Na comunidade CBSE, não existe consenso sobre um componente ser uma unidade independente de software que pode ser composta com outros componentes para criar um sistema de software. Além disso, as pessoas têm definições diferentes a respeito de um componente de software. Councill e Heineman (2001) definem um componente como:

> *Um elemento de software que está de acordo com um modelo de componente padrão e pode ser independentemente implantado e composto, sem modificações, de acordo com um padrão de composição.*

Essa definição é essencialmente baseada em padrões, assim, uma unidade de software que esteja em conformidade com esses padrões é um componente. No entanto, Szyperski (2002) não menciona padrões em sua definição de um componente, mas, em vez disso, concentra-se nas principais características dos componentes:

> *Um componente de software é uma unidade de composição com interfaces contratualmente especificadas e apenas dependências de contexto explícitas. Um componente de software pode ser implantado de forma independente e está sujeito a ser composto por parte de terceiros.*

Ambas as definições baseiam-se na noção de um componente como um elemento incluído em um sistema, em vez de um serviço que é referenciado pelo sistema. No entanto, elas também são compatíveis com a ideia de um serviço como um componente.

Szyperski também afirma que um componente não tem nenhum estado externamente observável. Isso significa que cópias de componentes são indistinguíveis. No entanto, alguns modelos de componentes, como o modelo de Enterprise Java Beans, permitem componentes com estado e, assim, estes não correspondem à definição de Szyperski. Embora componentes sem estado certamente sejam mais simples de usar, existem alguns sistemas em que os componentes com estado são mais convenientes e reduzem a complexidade do sistema.

O que as definições anteriormente mencionadas têm em comum é o fato de concordarem que os componentes são independentes e que, em um sistema, eles são a unidade fundamental de composição. Em minha opinião, uma melhor definição de um componente pode ser obtida combinando-se essas propostas. A Tabela 17.1 mostra o que eu considero que sejam as características essenciais de um componente como na CBSE.

Tabela 17.1 Características dos componentes

Característica do componente	Descrição
Padronizado	A padronização de componentes significa que um componente usado em um processo CBSE precisa obedecer a um modelo de componentes padrão. Esse modelo pode definir as interfaces de componentes, metadados de componente, documentação, composição e implantação.
Independente	Um componente deve ser independente, deve ser possível compor e implantá-lo sem precisar usar outros componentes específicos. Nessas situações, em que o componente precisa dos serviços prestados externamente, estes devem ser explicitamente definidos em uma especificação de interface 'requires'.
Passível de composição	Para um componente ser composto, todas as interações externas devem ter lugar por meio de interfaces publicamente definidas. Além disso, ele deve proporcionar acesso externo a informações sobre si próprio, como seus métodos e atributos.
Implantável	Para ser implantável, um componente dever ser autocontido. Deve ser capaz de operar como uma entidade autônoma em uma plataforma de componentes que forneça uma implementação do modelo de componentes, o que geralmente significa que o componente é binário e não tem como ser compilado antes de ser implantado. Se um componete é implantado como um serviço, ele não precisa ser implantado por um usuário de um componente. Pelo contrário, é implantado pelo prestador do serviço.
Documentado	Os componentes devem ser completamente documentados para que os potenciais usuários possam decidir se satisfazem a suas necessidades. A sintaxe e, idealmente, a semântica de todas as interfaces de componentes devem ser especificadas.

Uma maneira útil de se pensar sobre um componente é como um provedor de um ou mais serviços. Quando um sistema precisa de um serviço, ele chama um componente para fornecer esse serviço sem se preocupar sobre onde esse componente está em execução ou a linguagem de programação usada para desenvolvê-lo. Por exemplo, um componente de um sistema de biblioteca pode fornecer um serviço de pesquisa que permite aos usuários pesquisarem catálogos de diferentes bibliotecas. Um componente que converte de um formato gráfico para outro (por exemplo, TIFF para JPEG) fornece um serviço de conversão de dados etc.

A percepção de um componente como um provedor de serviços enfatiza duas características essenciais de um componente reusável:

1. O componente é uma entidade executável independente que é definida por suas interfaces. Para usá-lo, você não precisa de nenhum conhecimento de seu código-fonte. Ele pode ser referenciado como um serviço externo ou incluído diretamente em um programa.
2. Os serviços oferecidos por um componente são disponibilizados por meio de uma interface, e todas as interações se dão por meio dessa interface. A interface de componente é expressa em termos de operações parametrizadas e seu estado interno nunca é exposto.

Os componentes têm duas interfaces relacionadas, como mostrado na Figura 17.1. Essas interface refletem os serviços que o componente fornece e os serviços de que o componente necessita para funcionar corretamente:

- A interface 'provides' define os serviços prestados pelo componente. Essa interface, essencialmente, é API de componente. Ela define os métodos que podem ser chamados por um usuário do componente. Em um diagrama de componentes UML, a interface 'provides' para um componente é indicada por um círculo no final de uma linha a partir do ícone de componente.
- A interface 'requires' especifica quais serviços devem ser fornecidos por outros componentes no sistema se um componente deve funcionar corretamente. Se eles não estiverem disponíveis, o componente não funcionará. Isso não compromete a independência ou a capacidade de implantação de um componente, pois a interface 'requires' não define como esses serviços deverão ser prestados. Em UML, o símbolo de uma interface 'requires' é um semicírculo no final de uma linha a partir do ícone de componente. Observe que os ícones de interface 'provides' e 'requires' podem encaixar-se como uma bola e um soquete.

Para ilustrar essas interfaces, a Figura 17.2 mostra um modelo de um componente que foi projetado para coletar e reunir informações a partir de um vetor de sensores. Ele é executado autonomamente para coletar dados durante um período e, sob requisição, fornece dados agrupados para um componente que chama. A interface 'requires' inclui métodos para adicionar, remover, iniciar, parar e testar sensores. O método report retorna os dados do sensor que foram coletados e o método listAll fornece informações sobre os sensores conectados. Apesar de eu não ter mostrado isso aqui, esses métodos associaram parâmetros especificando os identificadores, locais de sensores, e assim por diante.

A interface 'requires' é usada para conectar os componentes aos sensores. Ela pressupõe que os sensores têm uma interface de dados, acessados por meio de sensorData, e uma interface de gerenciamento, acessada por meio de sensorManagement. Essa interface foi projetada para conectar-se a diferentes tipos de sensores, assim ela não inclui operações de sensores específicos, como Test, provideReading etc. Em vez disso, os comandos usados por um tipo específico de sensores são embutido sem uma *string*, que é um parâmetro para as operações na interface 'requires'. Os componentes adaptadores analisam essa *string* e traduzem os comandos embutidos para a interface de controle específica de cada tipo de sensor. Neste capítulo eu dicuto o uso de adaptadores, quando mostro como o componente coletor de dados está ligado a um sensor (Figura 17.10).

Uma diferença fundamental entre um componente como um serviço externo e um componente como um elemento de programa é que os serviços são entidades totalmente independentes. Eles não têm uma interface 'requires'. Diferentes programas podem usar esses serviços sem a necessidade de implementar qualquer suporte adicional exigido pelo serviço.

Figura 17.1 Interfaces de componentes

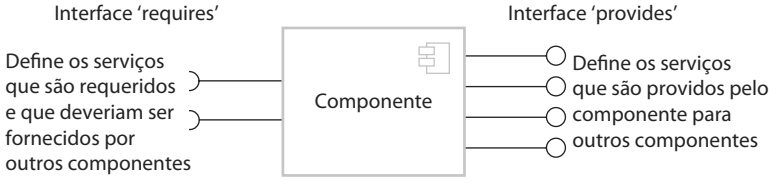
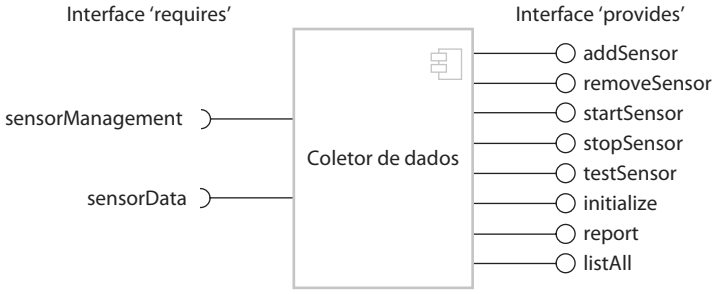


Figura 17.2 Um modelo de componente coletor de dados



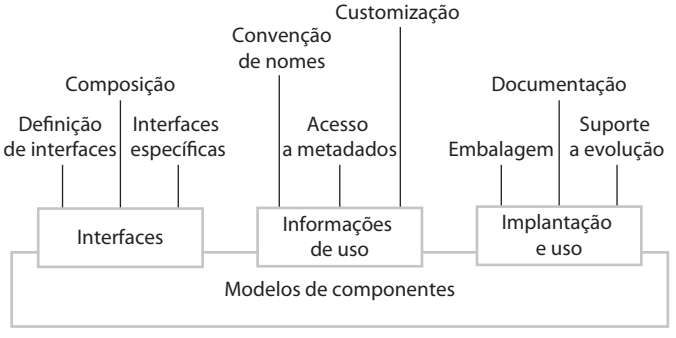
17.1.1 Modelos de componentes

Um modelo de componente é uma definição de normas para a implementação, documentação e implantação de componentes. Essas normas servem para os desenvolvedores de componentes garantirem que os componentes podem interoperar. Elas também existem para os fornecedores de infraestruturas de execução de componentes que oferecem suporte à operação de componentes. Muitos modelos de componentes foram propostos, mas, atualmente, os modelos mais importantes são o modelo WebServices, o modelo Enterprise Java Beans (EJB) da Sun e o modelo .NET da Microsoft (LAU e WANG, 2007).

Os elementos básicos de um modelo ideal de componentes são discutidos por Weinreich e Sametinger (2001). Eu sumarizo esses elementos de modelo na Figura 17.3, em um diagrama que mostra que os elementos de um modelo de componente definem as interfaces de componentes, as informações que você precisa para usar o componente em um programa e como um componente deve ser implantado.

1. *Interfaces*. Os componentes são definidos pela especificação de suas interfaces. O modelo de componente especifica como as interfaces devem ser definidas e os elementos, como nomes de operação, parâmetros e exceções, que devem ser incluídos na definição de interface. O modelo também deve especificar a linguagem usada para definir as interfaces de componente. Para *web services*, é o WSDL (do inglês Web Services Description Language), que discuto no Capítulo 19; o EJB é específico de Java, então o Java é usado como a linguagem de definição de interface; no .NET, as interfaces são definidas usando-se a Linguagem Intermediária Comum (CIL, do inglês *Common Intermediate Language*). Alguns modelos de componentes exigem interfaces específicas que devem ser definidas por um componente. Esses modelos são usados para compor o componente com a infraestrutura de modelo de componente, que fornece serviços padronizados, como gerenciamento de proteção e transação.
2. *Uso*. Para que componentes sejam distribuídos e acessados remotamente, eles precisam ter um nome exclusivo ou identificador associado a eles. Isso deve ser globalmente exclusivo — por exemplo, no EJB, um nome hierárquico é gerado com a raiz baseada em um nome de domínio de Internet. Os serviços têm um único URI (*Uniform Resource Identifier*).

Figura 17.3 Elementos básicos de um modelo de componentes



Metadados de componentes são dados sobre o componente em si, como informações sobre suas interfaces e atributos. Os metadados são importantes porque permitem aos usuários do componente descobrirem quais serviços são providos e requeridos. Implementações do modelo de componente, normalmente, incluem maneiras específicas (como o uso de uma interface de reflexão em Java) para acessar esse componente metadados.

Os componentes são entidades genéricas e, quando implantados, precisam ser configurados para caber em um sistema de aplicação. Por exemplo, você poderia configurar o componente Data collector (Figura 17.1) definindo o número máximo de sensores em um vetor de sensores. O modelo de componente, portanto, pode especificar como os componentes binários podem ser customizados para um ambiente de implantação específico.

3. *Implantação*. O modelo de componente inclui uma especificação de como componentes devem ser empacotados para implantação como entidades independentes, executáveis. Como os componentes são entidades independentes, eles precisam ser empacotados com todos os softwares de suporte não fornecidos pela infraestrutura de componente ou não serão definidos em uma interface 'requires'. Informações de implantação incluem informações sobre o conteúdo de um pacote e sua organização binária.

Sempre que surgirem novos requisitos, inevitavelmente os componentes terão de ser alterados ou substituídos. O modelo de componentes, portanto, pode incluir regras que gorvernam quando e como é permitida a substituição de componentes. Finalmente, o modelo de componente pode definir a documentação do componente que deve ser produzida. Isso é usado para localizar o componente e decidir se ele é apropriado.

Para componentes implementados como unidades de programa, em vez de serviços externos, o modelo de componente define os serviços a serem fornecidos pelo *middleware* que ofereça suporte à execução de componentes. Weinreich e Sametinger (2001) usam a analogia de um sistema operacional para explicar modelos de componentes. Um sistema operacional fornece um conjunto de serviços genéricos que podem ser usados pelas aplicações. Uma implementação do modelo de componente fornece serviços compartilhados comparáveis para componentes. Figura 17.4 mostra alguns dos serviços que podem ser fornecidos por uma implementação de um modelo de componente.

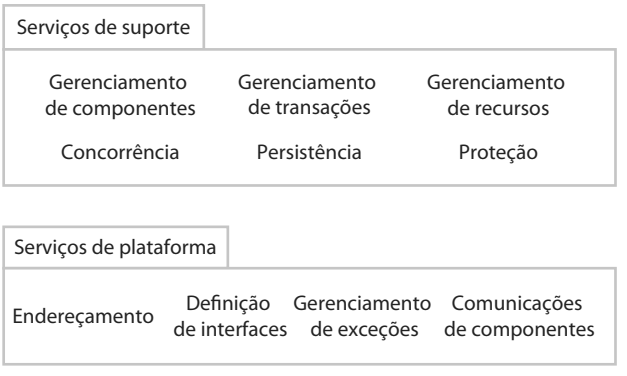
Os serviços fornecidos por uma implementação de modelo de componente dividem-se em duas categorias:

1. Serviços de plataforma, que permitem aos componentes se comunicarem e interagirem em um ambiente distribuído. Esses são os serviços fundamentais que devem estar disponíveis em todos os sistemas baseados em componentes.

2. Serviços de suporte, que são serviços comuns, suscetíveis de serem requeridos por muitos componentes diferentes. Por exemplo, muitos componentes requerem autenticação para garantir que o usuário dos serviços de componente seja autorizado. Faz sentido fornecer um conjunto-padrão de serviços de *middleware* para uso de todos os componentes, o que reduz os custos de desenvolvimento de componentes e as potenciais incompatibilidades de componentes podem ser evitadas.

O *middleware* implementa os serviços de componente e fornece interfaces para esses serviços. Para fazer uso dos serviços previstos por uma infraestrutura de modelo de componentes, você pode pensar em componentes para serem implantados em um 'contêiner'. Um contêiner é uma implementação dos serviços de suporte mais uma definição das interfaces que um componente deve fornecer para integrá-lo com o contêiner. Incluir um componente no contêiner significa que o componente pode acessar os serviços de suporte e o contêiner pode

Figura 17.4 Serviços de *middleware* definidos em um modelo de componente



acessar as interfaces de componente. Quando em uso, as interfaces de componentes próprios não são acessadas diretamente por outros componentes. Em vez disso, elas são acessadas por meio de uma interface de contêiner que chama código para acessar a interface de componente embutido.

Os contêiners são grandes e complexos e, quando você implanta um componente em um contêiner, você tem acesso a todos os serviços de *middleware*. No entanto, os componentes simples podem não precisar de todas as facilidades oferecidas pelo suporte de *middleware*. A abordagem adotada em *web services* para prestação de serviços comuns, portanto, é um pouco diferente. Para *web services*, padrões foram definidos para serviços comuns, como gerenciamento de transações e proteção, e esses padrões têm sido implementados como bibliotecas de programa. Caso você esteja implementando um componente de serviço, use apenas os serviços comuns de que você necessita.

17.2 Processos CBSE

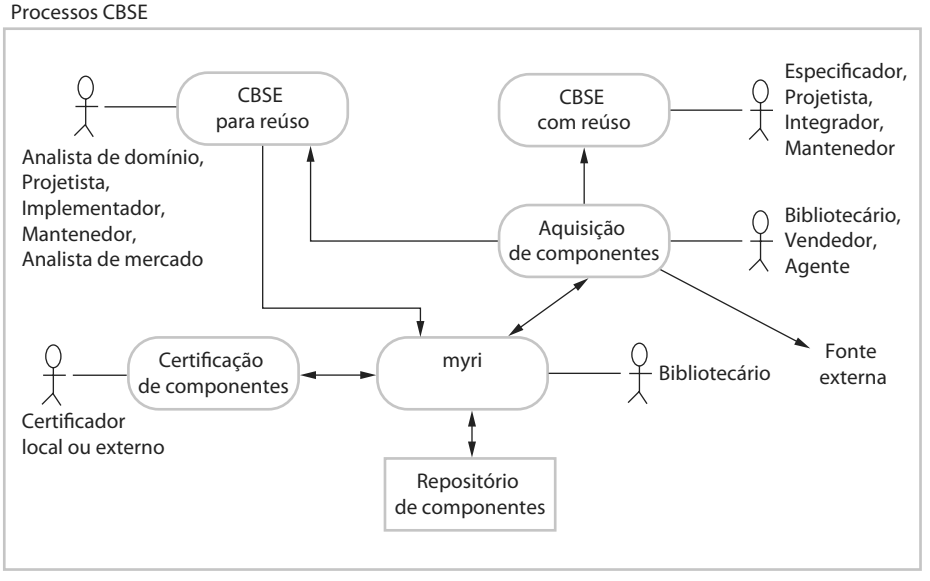
Os processos CBSE são processos de software que oferecem suporte a engenharia de software baseada em componentes. Consideram as possibilidades de reúso e as diferentes atividades do processo envolvidas no desenvolvimento e uso de componentes reusáveis. A Figura 17.5 (KOTONYA, 2003) apresenta uma visão geral dos processos CBSE. No nível mais alto, existem dois tipos de processos CBSE:

1. *Desenvolvimento para reúso*. Esse processo está interessado no desenvolvimento de componentes ou serviços que serão reusados em outras aplicações. Esse processo geralmente envolve generalizar os componentes existentes.
2. *Desenvolvimento com reúso*. Esse é o processo de desenvolvimento de novas aplicações usando componentes e serviços existentes.

Esses processos têm objetivos diferentes e, portanto, incluem atividades diferentes. No desenvolvimento por processo de reúso, o objetivo é produzir um ou mais componentes reusáveis. Você conhece os componentes com os quais trabalhará, além de ter acesso a seu código-fonte para generalizá-lo. Em desenvolvimento com reúso, você não sabe quais componentes estão disponíveis, por isso você precisa descobrir esses componentes e projetar seu sistema para fazer o uso mais eficiente deles. Você não pode ter acesso ao código-fonte do componente.

Na Figura 17.5, você pode ver que os processos básicos CBSE com e para reúso apoiam os processos que estão preocupados com a aquisição de componente, gerenciamento de componente e certificação de componente:

Figura 17.5 Processos CBSE



1. Aquisição de componente é o processo de aquisição de componentes para reúso ou desenvolvimento em um componente reusável. Pode envolver acesso a componentes desenvolvidos localmente ou serviços, ou encontrar esses componentes em uma fonte externa.
2. O gerenciamento de componente está preocupado com o gerenciamento de componentes reusáveis da empresa, garantindo que eles sejam devidamente catalogados, armazenados e disponibilizados para reúso.
3. A certificação do componente é o processo de verificação e certificação de que um componente atende a sua especificação.

Os componentes mantidos por uma organização podem ser armazenados em um repositório de componentes que inclui os componentes e as informações sobre seu uso.

17.2.1 CBSE para reúso

CBSE para reúso é o processo de desenvolver componentes reusáveis e torná-los disponíveis para reúso por meio de um sistema de gerenciamento de componentes. A visão dos primeiros defensores da CBSE (SZYPERSKI, 2002) foi a de que se desenvolveria um próspero mercado de componentes. Haveria provedores de componentes especializados e fornecedores de componentes que organizariam a venda de componentes de diferentes desenvolvedores. Os desenvolvedores de software comprariam componentes para incluir em um sistema ou pagar por serviços enquanto estes fossem usados. No entanto, essa visão não se concretizou. Relativamente, existem poucos fornecedores de componentes e a compra deles é incomum. Até o momento, o mercado de serviços também é pouco desenvolvido, apesar das previsões de significativa expansão nos próximos anos.

Por conseguinte, é mais provável que CBSE para reúso ocorra em uma organização que tenha o compromisso de usar a engenharia de software orientada a reúso. Eles pretendem explorar os ativos de software que foram desenvolvidos em diferentes partes da empresa. No entanto, esses componentes desenvolvidos internamente em geral não são reusáveis sem alterações. Muitas vezes, eles incluem recursos específicos da aplicação e interfaces que não são suscetíveis de serem necessárias em outros programas nos quais o componente seja reusado.

Para fazer componentes reusáveis você precisa se adaptar e estender os componentes específicos da aplicação para criar versões mais genéricas e, portanto, mais reusáveis. Certamente, essa adaptação tem um custo associado. Assim, você precisa decidir, em primeiro lugar, se um componente é suscetível de ser reusado, e em segundo lugar, se a economia de custos do reúso futuro justifica os custos de fazer o componente reusável.

Para responder à primeira dessas perguntas você precisa decidir se o componente implementa uma ou mais abstrações de domínio estável. As abstrações de domínio estável são elementos fundamentais de domínio da aplicação, os quais mudam lentamente. Por exemplo, em um sistema bancário, abstrações de domínio podem incluir contas, titulares de contas e demonstrações. Em um sistema de gerenciamento hospitalar, abstrações de domínio podem incluir pacientes, tratamentos e enfermeiros. Essas abstrações de domínio, às vezes, são chamadas 'objetos de negócios'. Se o componente é uma implementação de uma abstração de domínio usada com frequência ou grupo de objetos de negócios relacionados, ele provavelmente pode ser reusado.

Para responder à pergunta sobre a relação custo-benefício você deve avaliar os custos das mudanças necessárias para tornar o componente reusável. Tratam-se dos custos da documentação de componente, validação de componente e referentes a tornar o componente mais genérico. As alterações necessárias a um componente para torná-lo mais reusável incluem:

- remoção de métodos específicos de aplicação;
- alteração de nomes para torná-los mais gerais;
- adicionar métodos para fornecer uma cobertura funcional mais completa;
- tornar o tratamento de exceções consistente para todos os métodos;
- adicionar uma interface 'configuração' para permitir que o componente possa ser adaptado a diferentes situações de uso;
- integrar componentes necessários para aumentar a independência.

O problema de tratamento de exceções é particularmente difícil. Os componentes devem, eles próprios, tratar exceções, porque cada aplicação terá seus próprios requisitos para tratamento de exceção. Em vez disso, o componente deve definir quais exceções podem surgir e publicá-las como parte da interface. Por exemplo, um componente simples que implemente uma estrutura de dados de pilha deve detectar e publicar exceções de *overflow* e *underflow* de pilha. No entanto, na prática, existem dois problemas com isso:

1. Publicar todas as exceções leva a interfaces inchadas, que são mais difíceis de serem compreendidas. Isso pode afastar os potenciais usuários do componente.
2. O funcionamento do componente pode depender de tratamento de exceção local e mudar isso pode ter sérias implicações para a funcionalidade do componente.

Mili et al. (2002) discutem formas de estimar os custos de fazer um componente reusável, bem como os retornos do investimento. Os benefícios de reusar, em vez de desenvolver um componente, não são apenas os ganhos de produtividade. Também existem ganhos de qualidade, porque um componente reusado deve ser mais confiável, além dos ganhos de tempo de entrada no mercado. Esses são elementos que aumentam o retorno que acumula com a implantação do software mais rapidamente. Mili et al. (2002) apresentam várias fórmulas para calcular esses ganhos, como faz o modelo COCOMO, discutido no Capítulo 23 (BOEHM et al., 2000). No entanto, os parâmetros dessas fórmulas são difíceis de serem estimados com precisão, e as fórmulas devem ser adaptadas às circunstâncias locais, o que dificulta seu uso. Suspeito que alguns gerentes de projeto de software usem esses modelos para estimar o retorno sobre o investimento a partir da reusabilidade de componentes.

Certamente, a possibilidade de reúso de um componente depende de seu domínio de aplicação e funcionalidade. À medida que você adiciona generalidades a um componente, você aumenta sua capacidade de reúso. No entanto, isso normalmente significa que o componente tem mais operações e é mais complexo, o que torna o componente mais difícil de ser entendido e usado.

Portanto, existe um compromisso inevitável entre o reúso e a usabilidade de um componente. Para tornar um componente reusável, você deve fornecer um conjunto de interfaces genéricas com operações que atendam a todas as possibilidades de uso de um componente. Tornar o componente usável significa fornecer uma interface simples, mínima, que seja de fácil compreensão. A reusabilidade adiciona complexidade e, portanto, reduz a compreensibilidade do componente. Por conseguinte, é mais difícil decidir quando e como reusar esse componente. Ao projetar um componente reusável, você deve, portanto, encontrar um equilíbrio entre a generalidade e a compreensibilidade.

Uma potencial fonte de componentes é a existência de sistemas legados. Conforme discutido no Capítulo 9, tratam-se de sistemas que cumprem uma importante função de negócios, mas que são escritos usando-se tecnologias obsoletas de software. Por causa disso, pode ser difícil usá-los com novos sistemas. No entanto, se você converter esses sistemas antigos para componentes, sua funcionalidade pode ser reusada em novas aplicações.

Esses sistemas legados em geral não definem claramente interfaces 'requires' e 'provides'. Para tornar esses componentes reusáveis, você deve criar um empacotador que defina as interfaces de componente. O empacotador oculta a complexidade do código subjacente e fornece uma interface para componentes externos acessarem os serviços fornecidos. Embora esse empacotador seja um software bastante complexo, o custo de seu desenvolvimento é, frequentemente, muito menor que o custo de reimplementação do sistema legado. Eu discuto essa abordagem mais detalhadamente no Capítulo 19, no qual explico como os recursos em um sistema legado podem ser acessados por meio de serviços.

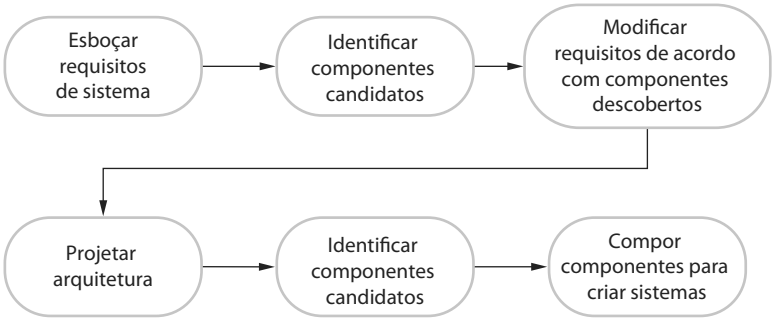
Depois de ter sido desenvolvido e testado um componente ou serviço reusável, este deve ser gerenciado para reúso futuro. O gerenciamento envolve decidir como classificar o componente para que ele possa ser descoberto, de modo que o componente fique disponível tanto em um repositório como em um serviço, que informações sobre o uso do componente sejam mantidas e que o controle sobre diferentes versões do componente seja preservado. Se o componente for *open source*, você pode torná-lo disponível em um repositório público, tal como Sourceforge. Se ele se destinar para uso em uma empresa, você poderá usar um sistema de repositório interno.

Uma empresa com um programa de reúso pode realizar alguma forma de certificação de componente antes que o componente seja disponibilizado para reúso. A certificação significa que alguém diferente do desenvolvedor verifica a qualidade do componente. Eles testam o componente e certificam se ele atingiu um padrão de qualidade aceitável, antes de ser disponibilizado para reúso. No entanto, isso pode ser um processo caro e muitas empresas simplesmente deixam os testes de qualidade para os desenvolvedores de componentes.

17.2.2 CBSE com reúso

O reúso bem-sucedido de componentes requer um processo de desenvolvimento sob medida para CBSE. A CBSE com o processo de reúso precisa incluir atividades que encontrem e integrem componentes reusáveis. A estrutura desse processo foi discutida no Capítulo 2. A Figura 17.6 mostra as principais atividades dentro desse processo. Algumas delas, como a descoberta inicial de requisitos de usuário, são executadas da mesma forma

Figura 17.6 CBSE com reúso



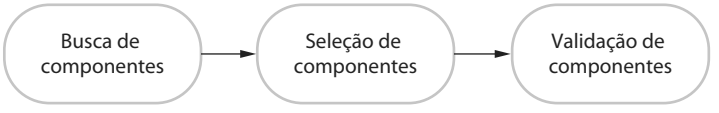
como em outros processos de software. No entanto, as diferenças essenciais entre CBSE com reúso e processos de software para desenvolvimento de software original são:

1. Os requisitos de usuário são inicialmente desenvolvidos em alto nível, em vez de detalhes, e os *stakeholders* são incentivados a serem tão flexíveis quanto possível na definição de seus requisitos. Os requisitos muito específicos limitam o número de componentes que poderiam atender a esses requisitos. No entanto, ao contrário do desenvolvimento incremental, você precisa de um conjunto completo de requisitos para poder identificar o maior número possível de componentes para reúso.
2. Requisitos são refinados e modificados no processo de acordo com os componentes disponíveis. Se os requisitos de usuário não podem ser satisfeitos por componentes disponíveis, você deve discutir os requisitos relacionados que podem ser suportados. Os usuários podem estar dispostos a mudar de opinião se isso significar a entrega mais barata ou mais rápida do sistema.
3. Após a arquitetura de sistema ser projetada, existe uma atividade adicional de refinamento da pesquisa e projeto de componente. Alguns componentes aparentemente usáveis podem vir a ser impróprios ou a não funcionar corretamente com outros componentes escolhidos. Embora a Figura 17.6 não mostre, isso implica que podem ser necessárias mudanças nos requisitos adicionais.
4. O desenvolvimento é um processo de composição em que os componentes descobertos são integrados. Trata-se de integrar os componentes com a infraestrutura de modelo de componente e, muitas vezes, desenvolver adaptadores que conciliem as interfaces dos componentes incompatíveis. Claro que funcionalidade adicional pode ser necessária acima e por cima daquelas fornecidas pelos componentes reusáveis.

O estágio de projeto de arquitetura é particularmente importante. Jacobson et al. (1997) descobriram que definir uma arquitetura robusta é crítico para reúso com êxito. Durante a atividade de projeto de arquitetura, você pode escolher um modelo de componente e uma plataforma de implementação. No entanto, muitas empresas têm uma plataforma de desenvolvimento padrão (por exemplo, .NET), assim o modelo de componente é predeterminado. Conforme discutido no Capítulo 6, nesse estágio você também estabelece a organização de alto nível do sistema e toma decisões sobre a distribuição e o controle de sistema.

Uma atividade exclusiva para o processo CBSE é identificar componentes ou serviços candidatos ao reúso. Isso envolve uma série de subatividades, como mostrado na Figura 17.7. Inicialmente, seu foco deve ser a busca e seleção. Você precisa convencer a si mesmo de que existem componentes disponíveis para atender a seus requisitos. Certamente, você deve fazer alguma verificação inicial de que o componente seja adequado, mas testes detalhados podem não ser necessários. Em estágios posteriores, depois que a arquitetura do sistema tenha sido projetada, você deve despender mais tempo na validação de componente. É necessário estar confiante de que os componentes identificados são realmente adequados para sua aplicação. Se não, você precisa repetir os processos de busca e seleção.

Figura 17.7 O processo de identificação de componentes



O primeiro passo na identificação de componentes é olhar para os componentes ou fornecedores confiáveis disponíveis localmente. Como eu disse na seção anterior, existem, relativamente, poucos fornecedores de componentes e, portanto, é possível que você procure por componentes que foram desenvolvidos em sua própria empresa. As empresas de desenvolvimento de software podem construir sua própria base de dados de componentes reusáveis, sem os riscos inerentes ao uso de componentes de fornecedores externos. Como alternativa, você pode decidir procurar bibliotecas de código disponíveis na Web, como o Sourceforge ou Google Code, para ver se o código-fonte para o componente que você precisa está disponível. Se você estiver procurando por serviços, existem inúmeros motores de busca na Web especializados capazes de descobrir *web services* públicos.

Uma vez que o processo de busca de componentes tenha identificado possíveis componentes, você deve selecionar componentes candidatos para avaliação. Em alguns casos, essa será uma tarefa simples; os componentes da lista implementarão os requisitos de usuário e não haverá competição entre os componentes que correspondam a esses requisitos. Em outros casos, no entanto, o processo de seleção poderá ser muito mais complexo; não haverá um mapeamento claro de requisitos para componentes e você poderá achar que vários componentes precisam ser integrados para atender a um requisito específico ou a um grupo de requisitos. Por isso, terá de decidir quais composições de componentes fornecem a melhor cobertura dos requisitos.

Uma vez que você tenha selecionado componentes para uma possível inclusão em um sistema, você deve validá-los para verificar se eles se comportam como anunciado. A extensão da validação necessária depende da fonte dos componentes. Se estiver usando um componente que foi desenvolvido por uma fonte confiável e conhecida, você pode decidir que o teste de componente é desnecessário. Você testa o componente apenas quando ele é integrado com outros componentes. Por outro lado, se você estiver usando um componente de origem desconhecida, deve sempre verificar e testar esse componente antes de o incluir em seu sistema.

A validação de componente envolve o desenvolvimento de um conjunto de casos de teste para um componente (ou, possivelmente, estendendo os casos de teste fornecidos com esse componente) e o desenvolvimento de um equipamento de teste para executar testes de componentes. O grande problema com validação de componente é que a especificação de componente pode não ser suficientemente detalhada para permitir que você desenvolva um conjunto completo de testes de componentes. Geralmente, os componentes são especificados informalmente, sendo sua especificação de interface a única documentação formal . Isso pode não incluir informações suficientes para você desenvolver um conjunto completo de testes que o convenceria de que a interface do componente anunciado é o que você precisa.

Além de testar se um componente para reúso faz o que você precisa, você também deve verificar se o componente não inclui qualquer código malicioso ou funcionalidade de que você não precisa. Os desenvolvedores profissionais raramente usam componentes de fontes não confiáveis, especialmente se essas fontes não fornecem o código-fonte. Portanto, o problema de código malicioso não é comum. No entanto, componentes podem, muitas vezes, conter funcionalidade que você não precisa e você deve verificar se essa funcionalidade não interferirá com o uso do componente.

O problema com funcionalidade desnecessária é que ela pode ser ativada pelo próprio componente, o que pode retardá-lo, levá-lo a produzir resultados surpreendentes ou, em alguns casos, causar falhas graves do sistema. O Quadro 17.1 resume uma situação na qual funcionalidade desnecessária em um sistema reusado causou uma falha catastrófica de software.

Quadro 17.1 Um exempo de falha de validação com software reusado

Ao desenvolver o lançador do Ariane 5, os projetistas decidiram reusar o software de referência inercial que teve sucesso no lançador do Ariane 4. O software de referência inercial mantém a estabilidade do foguete. Eles decidiram reusar esse software sem alterações (como você faria com componentes), embora tenham incluído uma funcionalidade adicional, a qual não foi exigida no Ariane 5.

No primeiro lançamento do Ariane 5, o software de navegação inercial falhou e o foguete não pode ser controlado. Os controladores em terra instruíram o foguete a se autodestruir e sua carga foi destruída.

A causa do problema foi uma exceção não tratada quando ocorreu uma conversão de um número de ponto fixo para um número inteiro que resultou em um *overflow* numérico. Isso levou o sistema de *run-time* a desligar o sistema de referência inercial e a estabilidade do foguete não pôde ser mantida.

Essa falha nunca aconteceu no Ariane 4, porque este tinha motores menos potentes e o valor convertido não pôde ser grande o suficiente para a conversão causar *overflow*.

O defeito aconteceu em um código que não era necessário no Ariane 5. Os testes de validação para o reúso de software basearam-se nos requisitos do Ariane 5. Como não havia requisito para a função que falhou, não foram realizados testes. Consequentemente, o problema com o software nunca foi descoberto nos testes de simulação de lançamento.

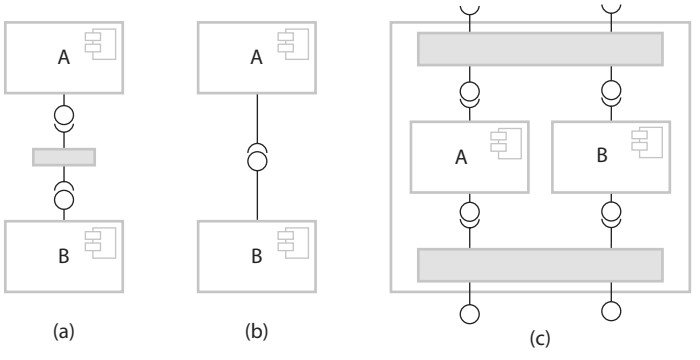
O problema no lançador do Ariane 5 surgiu porque as suposições feitas sobre o software para o Ariane 4 eram inválidas para o Ariane 5. Esse é um problema geral com componentes reusáveis. Originalmente, eles são implementados para um ambiente de aplicações e incorporam pressupostos sobre esse ambiente. Esses pressupostos raramente são documentados, então, quando o componente é reusado, é impossível derivar testes para verificar se os pressupostos ainda são válidos. Se você vai reusar um componente em um ambiente diferente, é possível que não descubra os pressupostos de ambientes embutidos até que use o componente em um sistema operacional.

17.3 Composição de componentes

A composição de componentes é o processo de integração de componentes uns com os outros e com o componente especialmente escrito '*glue code*' para criar um sistema ou outro componente. Existem várias maneiras pelas quais você pode compor componentes, como mostrado na Figura 17.8. Da esquerda para a direita, esses diagramas ilustram a composição sequencial, a composição hierárquica e a composição aditiva. Na discussão a seguir, eu assumo que você está compondo dois componentes (A e B) para criar um novo componente:

1. Na Figura 17.8, a composição sequencial está apresentada no item (a). Você cria um novo componente a partir de dois componentes existentes, por chamar os componentes existentes em sequência. Você pode pensar a composição como uma composição de 'interfaces provides'. Ou seja, os serviços oferecidos pelo componente A são chamados e os resultados retornados por A são usados na chamada para os serviços oferecidos pelo componente B. Os componentes não chamam uns aos outros na composição sequencial. Algum '*glue code*' extra é necessário para chamar os serviços de componente na ordem certa e garantir que os resultados entregues por componente sejam compatíveis com as entradas esperadas pelo componente B. A interface 'provides' da composição depende da funcionalidade combinada de A e B, mas, normalmente, essa não será uma composição de suas interfaces 'provides'. Esse tipo de composição pode ser usado com componentes que são elementos de programa ou componentes que são serviços.
2. Na Figura 17.8, a composição hierárquica está apresentada no item (b). Esse tipo de composição ocorre quando um componente chama diretamente os serviços prestados por outro componente. O componente chamado fornece os serviços necessários para o componente chamador. Portanto, a interface 'provides' do componente chamado deve ser compatível com a interface 'requires' do componente chamador. O componente A chama diretamente o componente B e, se suas interfaces corresponderem, pode não haver necessidade de código adicional. No entanto, se houver uma incompatibilidade entre a interface 'requires' de A e a interface 'provides' de B, então algum código de conversão pode ser necessário. Como serviços não têm uma interface 'requires', esse modo de composição não é usado quando componentes são implementados como *web services*.
3. A composição aditiva corresponde à situação (c) na Figura 17.8. Ela ocorre quando dois ou mais componentes são colocados juntos (adicionados) para se criar um novo componente, que combina suas funcionalidades. A interface 'provides' e a interface 'requires' do novo componente é uma combinação das interfaces correspondentes nos componentes A e B. Os componentes são chamados separadamente por meio da interface externa do componente composto. A e B não são dependentes e não chamam uns aos outros. Esse tipo de composição pode ser usado com componentes que são unidades de programas ou componentes que são serviços.

Figura 17.8 Tipos de composição de componentes



Você pode usar todas as formas de composição de componentes durante a criação de um sistema. Em todos os casos, talvez seja necessário escrever um '*glue code*' para ligar os componentes. Por exemplo, para uma composição sequencial, a saída do componente A normalmente se torna a entrada do componente B. Você precisa de instruções intermediárias que chamem o componente A, coletem o resultado e, em seguida, chamem o componente B, com esse resultado como um parâmetro. Quando um componente chama outro, talvez você precise introduzir um componente intermediário que garanta que a interface 'provides' e a interface 'requires' sejam compatíveis.

Quando você escreve novos componentes especialmente para a composição, você deve criar as interfaces desses componentes de maneira que sejam compatíveis com outros componentes do sistema. Portanto, você pode compor esses componentes facilmente em uma única unidade. No entanto, quando os componentes são desenvolvidos para reúso de forma independente, você frequentemente é confrontado com as incompatibilidades de interfaces. Isso significa que as interfaces dos componentes que você deseja compor não são as mesmas. Podem ocorrer três tipos de incompatibilidades:

1. *Incompatibilidade de parâmetro*. As operações de cada lado da interface têm o mesmo nome, mas com tipos de parâmetro ou número de parâmetros diferentes.
2. *Incompatibilidade de operação*. Os nomes das operações nas interfaces 'provides' e 'requires' são diferentes.
3. *Incompletude de operação*. O funcionamento da interface 'provides' de um componente é um subconjunto da interface 'requires' de outro componente ou vice-versa.

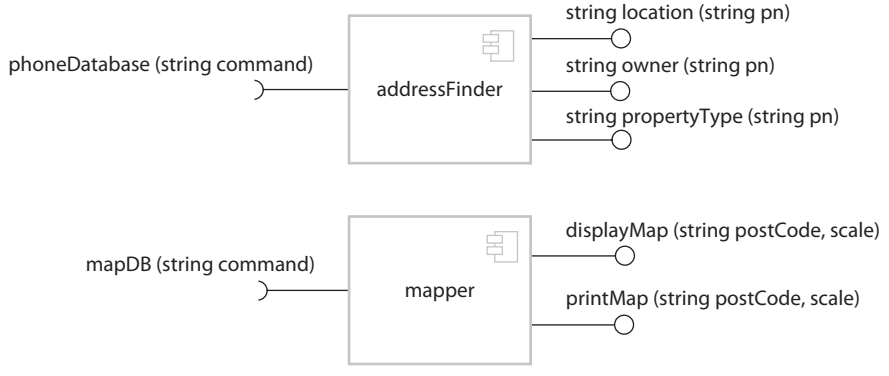
Em todos os casos, você pode resolver o problema da incompatibilidade escrevendo um adaptador que reconcilia as interfaces dos dois componentes reusados. Um componente adaptador converte uma interface para outra. A forma exata do adaptador depende do tipo de composição. Às vezes, como no exemplo a seguir, o adaptador leva um resultado de um componente e converte em um formulário, que pode ser usado como entrada para outro. Em outros casos, o adaptador pode ser chamado pelo componente A como um *proxy* para o componente B. Essa situação ocorre se A pretende chamar B, mas os detalhes da interface 'requires' de A não coincidem com os detalhes da interface 'provides' de B. O adaptador reconcilia essas diferenças, convertendo os parâmetros de entrada de A para os parâmetros de entrada requeridos por B. Ele, em seguida, chama B para fornecer os serviços requeridos por A.

Para ilustrar os adaptadores, considere os dois componentes mostrados na Figura 17.9, cujas interfaces são incompatíveis. Eles podem ser parte de um sistema usado pelos serviços de emergência. Quando o operador de emergência recebe uma chamada, o número do telefone é a entrada para o componente `addressFinder` localizar o endereço. Em seguida, usando o componente mapper, o operador imprime um mapa para ser enviado para o veículo, despachado para a situação de emergência. Na verdade, os componentes teriam interfaces mais complexas do que as mostradas aqui, mas a versão simplificada ilustra o conceito de um adaptador.

O primeiro componente, `addressFinder`, localiza o endereço que corresponde a um número de telefone. Ele também pode retornar o dono da propriedade associado ao número de telefone e tipo de propriedade. O componente `mapper` leva um código postal (nos Estados Unidos, um código postal padrão com os quatro dígitos adicionais identificando o local da propriedade) e exibe ou imprime um mapa das ruas da área em torno desse código em uma escala especificada.

Em princípio, esses componentes são compostos porque o local da propriedade inclui o código postal, ou CEP. No entanto, você precisa escrever um componente adaptador chamado `postCodeStripper` que leva os dados

Figura 17.9 Componentes com interfaces incompatíveis



do local do addressFinder e remove o código postal. Então, esse código postal é usado como entrada para o mapper, e o mapa das ruas é exibido em uma escala de 1:10.000. O código a seguir, que é um exemplo de composição sequencial, ilustra a sequência de chamadas que é requerida para implementar isto:

```
address = addressFinder.location (phonenumber) ;

postCode = postCodeStripper.getPostCode (address) ;

mapper.displayMap(postCode, 10 000) ;
```

Outro caso em que um componente adaptador pode ser usado na composição hierárquica é quando um componente pretende fazer uso de outro, mas há uma incompatibilidade entre a interface 'provides' e a interface 'requires' dos componentes na composição. Na Figura 17.10, eu ilustro o uso de um adaptador; no caso, um adaptador é usado para conectar um componente coletor de dados e um sensor. Eles poderiam ser usados na implementação de um sistema de estação meteorológica no deserto, conforme discutido no Capítulo 7.

Os componentes coletor de dados e sensor são compostos usando um adaptador que reconcilia a interface 'requires' do componente coletor de dados com a interface 'provides' do componente sensor. O componente coletor de dados foi projetado com uma interface genérica 'requires' que suporta a coleta de dados e o gerenciamento de sensor. Para cada uma dessas operações, o parâmetro é uma *string* de texto que representa os comandos específicos do sensor. Por exemplo, para emitir um comando de coleta, você diria `sensorData`('collect'). Como mostrado na Figura 17.10, o próprio sensor tem operações distintas, como iniciar, parar e coletar dados.

O adaptador analisa a *string* de entrada, identifica o comando (por exemplo, coletar) e, em seguida, chama o `Sensor.getdata` para coletar o valor do sensor. Em seguida, ele retorna o resultado (como uma *string* de caracteres) para o componente coletor de dados. Esse estilo de interface significa que o coletor de dados pode interagir com diferentes tipos de sensores. Um adaptador separado, que converte os comandos de sensor a partir do `Datacollector` para a interface de sensor real, é implementado para cada tipo de sensor.

Essa discussão, sobre composição de componentes, assume que você pode dizer, a partir da documentação do componente, se a interface é compatível ou não. Naturalmente, a definição de interface inclui os tipos de parâmetros e o nome da operação, para que você possa fazer alguma avaliação de sua compatibilidade. No entanto, você depende da documentação de componente para decidir se as interfaces são semanticamente compatíveis.

Para ilustrar esse problema, considere a composição mostrada na Figura 17.11. Esses componentes são usados para implementar um sistema que transfere imagens de uma câmera digital e as armazena em uma biblioteca de fotografias. O usuário do sistema pode fornecer informações adicionais para descrever e catalogar a fotografia. Para evitar a confusão, eu não mostro nesta obra todos os métodos de interface. Em vez disso, mostro simplesmente os métodos necessários para ilustrar o problema da documentação de componentes. Os métodos na interface da Biblioteca de Fotografias são:

```
public void addItem (Identifier pid ; Photograph p; CatalogEntry photodesc) ;

public Photograph retrieve (Identifier pid) ;

public CatalogEntry catEntry (Identifier pid) ;
```

Figura 17.10 Um adaptador conectando o coletor de dados e um sensor

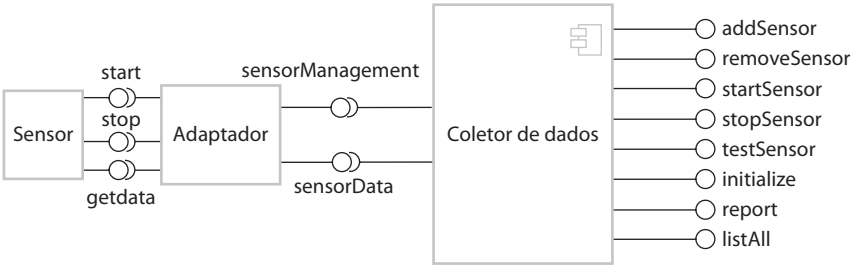
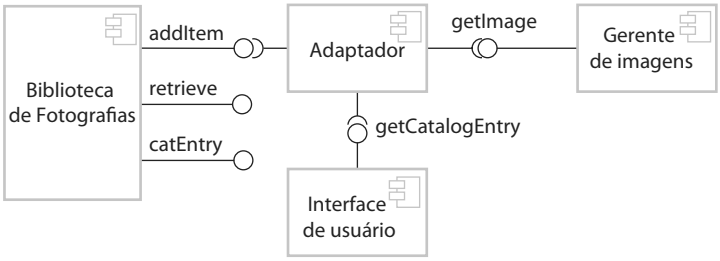


Figura 17.11 Composição de uma biblioteca de fotografias



Suponha que a documentação para o método addItem na Biblioteca da Fotografias seja:

> *Esse método adiciona uma fotografia para a biblioteca e associa o identificador de fotografia e o descritor de catálogo com a fotografia.*

Essa descrição parece explicar o que o componente faz, mas considere as seguintes perguntas:

- O que acontece se o identificador de fotografia já estiver associado com uma fotografia na biblioteca?
- O descritor de fotografia, assim como a fotografia, é associado com a entrada do catálogo? Ou seja, se você excluir a fotografia, também exclui as informações do catálogo?

Na descrição informal não existem informações suficientes de addItem para responder a essas perguntas. Naturalmente, é possível adicionar mais informações para a descrição da linguagem natural do método, mas em geral a melhor maneira de resolver as ambiguidades é usar uma linguagem formal para descrever a interface. A especificação mostrada no Quadro 17.2 é parte da descrição da interface da `Photo Library` que adiciona informações para a descrição informal.

A especificação no Quadro 17.2 usa pré e pós-condições definidas em uma notação baseada na linguagem de restrição de objeto (OCL, do inglês *object constraint language*), que faz parte da UML (WARMER e KLEPPE, 2003). A OCL é projetada para descrever restrições nos modelos de objeto da UML; permite expressar predicados que devem ser verdadeiros sempre, que devem ser verdadeiros antes que um método seja executado e que devem ser verdadeiros após um método ser executado. Essas são invariantes, pré-condições e pós-condições. Para acessar o valor de uma variável antes de uma operação, você adiciona @pré após seu nome. Portanto, usando a idade como um exemplo:

```
idade = idade@pre + 1
```

Essa declaração significa que o valor da idade, depois de uma operação, é um a mais do que era antes da operação.

As abordagens baseadas em OCL estão cada vez mais sendo usadas para adicionar informações semânticas aos modelos da UML e as descrições OCL podem ser usadas para dirigir geradores de códigos na engenharia orientada a modelos. A abordagem geral foi obtida a partir do Projeto por Contrato de Meyer (MEYER, 1992), em que as

Quadro 17.2 A descrição OCL da interface da `Photo Library`

```
- O contexto dos nomes-chave do componente ao qual as condições se aplicam ao contexto AddItem
- As pré-condições especificam o que deve ser verdadeiro antes da execução de addItem
pre:    PhotoLibrary.libSize( ) > 0
        PhotoLibrary.retrieve (pid) = null
- As pós-condições especificam o que é verdadeiro após a execução
post:   libSize ( ) = libSize( )@pre + 1
        PhotoLibrary.retrieve(pid), p =
        PhotoLibrary.catEntry(pid) = photodesc
        context delete
pre:    PhotoLibrary.retrieve (pid) < >null ;
post:   PhotoLibrary.retrieve(pid) = null
        PhotoLibrary.catEntry(pid) = PhotoLibrary.catEntry(pid)@pre
        PhotoLibrary.libSize( ) = libSize( )@pre-1
```

interfaces e as obrigações dos objetos que se comunicam são formalmente especificadas e impostas pelo sistema de *run-time*. Meyer sugere que o uso de Projeto por Contrato é essencial se quisermos desenvolver componentes confiáveis (MEYER, 2003).

O Quadro 17.2 inclui uma especificação para os métodos addItem e delete na `Photo Library`. O método que está sendo especificado é indicado pelo contexto de palavra-chave e as pré e pós-condições, pelas palavras-chave *pre* e *post*. As pré-condições para addItem afirmam que:

1. Não deve haver, na biblioteca, uma fotografia com o mesmo identificador que a fotografia que será inserida.
2. A biblioteca deve existir — assuma que ao criar uma biblioteca você deve adicionar a ela, pelo menos, um único item, de maneira que o tamanho de uma biblioteca seja sempre maior que zero.
3. As pós-condições para addItem afirmam que:

   O tamanho da biblioteca aumentou em um (tão somente uma única entrada foi feita).

   Se você buscar usando o mesmo identificador, você recebe de volta a fotografia que adicionou.

Se você pesquisar o catálogo usando esse identificador, você recebe de volta a entrada de catálogo que você fez.

A especificação de delete fornece mais informações. A pré-condição afirma que, para um item ser deletado, ele deve estar na biblioteca e que, após ser deletada, a foto já não pode ser recuperada e o tamanho da biblioteca será reduzido em um. No entanto, deletar a foto não deleta a entrada de catálogo — você ainda poderá recuperá-la depois de a foto ter sido deletada. A razão para isso é que você pode desejar manter, no catálogo, informações sobre por que uma foto foi excluída, seu novo local, e assim por diante.

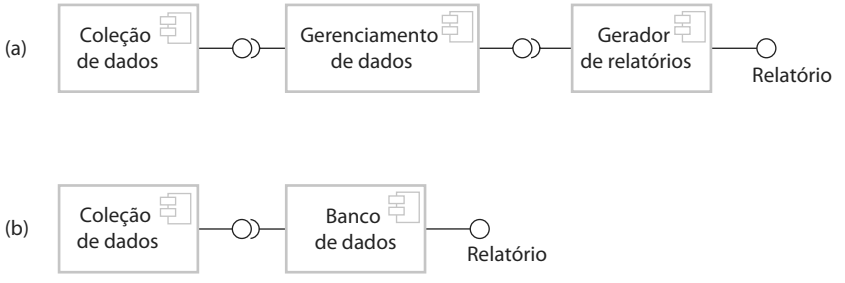
Ao criar um sistema de composição de componentes, você pode achar que não existem conflitos potenciais entre os requisitos funcionais e não funcionais, a necessidade de entregar um sistema tão rapidamente quanto possível e a necessidade de criar um sistema que possa evoluir de acordo com a mudança de requisitos. As decisões que você pode ter de levar em consideração são:

1. Qual composição de componentes é mais eficaz na entrega dos requisitos funcionais para o sistema?
2. Qual composição dos componentes tornará mais fácil adaptar os componentes compostos quando seus requisitos mudarem?
3. Quais serão as propriedades emergentes do sistema composto? Essas são propriedades como desempenho e confiabilidade. Você só pode avaliá-las depois que o sistema completo tenha sido implementado.

Infelizmente, existem muitas situações em que as soluções para os problemas de composição podem entrar em conflito. Por exemplo, considere uma situação como a ilustrada na Figura 17.12, em que um sistema pode ser criado por meio de duas composições alternativas. O sistema é uma coleção de dados e sistema de relatórios em que os dados são coletados de fontes diferentes e armazenados em um banco de dados e, em seguida, são produzidos relatórios diferentes desses dados.

Aqui, existe um potencial conflito entre o desempenho e a capacidade de adaptação. A composição (a) é mais adaptável, mas a composição (b) talvez seja mais rápida e mais confiável. As vantagens da composição (a) é que os relatórios e o gerenciamento de dados são separados e, assim, há mais flexibilidade para mudanças futuras. O sistema de gerenciamento de dados pode ser substituído e, se forem requisitados relatórios que o componente de relatório atual não pode produzir, esse componente também pode ser substituído sem a necessidade de alteração do componente de gerenciamento de dados.

Figura 17.12 Componentes da coleção de dados e geração de relatório



Na composição (b), um componente de banco de dados com recursos internos de geração de relatórios (por exemplo, o Microsoft Access) é usado. A principal vantagem da composição (b) é que existem menos componentes; assim, ela será implementada mais rapidamente, pois não há *overheads* de comunicação de componente. Além disso, as regras de integridade de dados que se aplicam ao banco de dados também serão aplicáveis aos relatórios. Esses relatórios não serão capazes de combinar dados de forma incorreta. Na composição (a) não existem essas restrições de modo que pudessem ocorrer erros nos relatórios.

Em geral, um princípio da boa composição é o princípio de separação de interesses. Ou seja, você deve tentar projetar seu sistema de forma que cada componente tenha seu papel claramente definido e que, idealmente, tais papéis não se sobreponham. No entanto, pode ser mais barato comprar um componente multifuncional, em vez de dois ou três componentes separados. Além disso, quando vários componentes são usados pode haver sanções na confiança ou no desempenho.

PONTOS IMPORTANTES

- A engenharia de software baseada em componentes é uma abordagem baseada no reúso para definição, implementação e composição de componentes independentes, fracamente acoplados em sistemas.
- Um componente é uma unidade de software cuja funcionalidade e dependências são completamente definidas por um conjunto de interfaces públicas. Os componentes podem ser compostos com outros componentes sem o conhecimento de sua implementação e podem ser implantados como uma unidade executável.
- Os componentes podem ser implementados como unidades de programa que são incluídas em um sistema ou como serviços externos que são referenciados a partir de dentro de um sistema.
- Um modelo de componente define um conjunto de padrões para componentes, incluindo normas de interface, normas de uso e normas de implantação. A implementação do modelo de componente fornece um conjunto de serviços comuns que podem ser usados por todos os componentes.
- Durante o processo CBSE, você precisa intercalar os processos de engenharia de requisitos e projeto de sistemas. Você precisa negociar os requisitos desejáveis pelos serviços que estão disponíveis a partir dos componentes reusáveis existentes.
- A composição do componente é o processo de 'conexão' dos componentes para a criação de um sistema. Os tipos de composições incluem a composição sequencial, a composição hierárquica e a composição aditiva.
- Ao compor componentes reusáveis que não foram escritos para sua aplicação, você talvez precise escrever adaptadores ou '*glue code*' para reconciliar as diferentes interfaces de componentes.
- Ao escolher as composições, você deve considerar a funcionalidade requerida para o sistema, os requisitos não funcionais e a facilidade com que um componente pode ser substituído quando o sistema é alterado.

LEITURA COMPLEMENTAR

Component-based Software Engineering: Putting the Pieces Together. Esse livro é uma coleção de artigos de vários autores sobre os diferentes aspectos da CBSE. Como todas as coleções, ela é um pouco variada, mas traz uma melhor discussão sobre as questões gerais de engenharia de software com componentes do que o livro do Szyperski. (HEINEMAN, G. T.; COUNCILL, W. T. *Component-based Software Engineering*: Putting the Pieces Together. Addison--Wesley, 2001.)

Component Software: Beyond Object-Oriented Programming, 2nd.ed. Essa edição atualizada do primeiro livro sobre CBSE abrange questões técnicas e não técnicas sobre CBSE. Ela traz mais detalhes das tecnologias específicas que o livro de Heineman e Councill, além de incluir uma discussão aprofundada das questões de mercado. (SZYPERSKI, C. *Component Software*: Beyond Object-Oriented Programming. 2. ed. Addison-Wesley, 2002.)

'Specification, Implementation and Deployment of Components'. Uma boa introdução aos fundamentos da CBSE. A mesma edição da *CACM* inclui artigos sobre componentes e desenvolvimento baseado em componentes. (CRNKOVIC, I.; HNICH, B.; JONSSON, T.; KIZILTAN, Z. *Comm. ACM*, v. 45, n. 10, out. 2002.) Disponível em: <http://dx.doi.org/10.1145/570907.570928>.

'Software Component Models'. Essa é uma discussão abrangente sobre os modelos de componentes comerciais e de pesquisa que classifica esses modelos e explica as diferenças entre eles. (LAU, K.-K.; WANG, Z. *IEEE Transactions on Software Engineering*, v. 33, n. 10, out. 2007.) Disponível em: <http://dx.doi.org/10.1109/TSE.2007.70726>.

EXERCÍCIOS

17.1 Por que é importante que todas as interações de componente sejam definidas por meio das interfaces 'requires' e 'provides'?

17.2 O princípio da independência de componentes significa que deveria ser possível substituir um componente por outro, implementado de maneira completamente diferente. Usando um exemplo, explique como tal substituição de componentes poderia ter consequências indesejadas e levar à falha de sistema.

17.3 Quais são as diferenças fundamentais entre componentes como elementos de programa e componentes como serviços?

17.4 Por que é importante que os componentes se baseiem em um modelo de componente padrão?

17.5 Usando um exemplo de um componente que implementa um tipo abstrato de dado, como uma pilha ou uma lista, mostre por que, geralmente, é necessário estender e adaptar componentes para o reúso.

17.6 Explique por que é difícil validar um componente reusável sem o código-fonte do componente. De que maneiras uma especificação formal de componentes simplificaria os problemas de validação?

17.7 Projete a interface 'provides' e a interface 'requires' de um componente reusável que possa ser usado para representar um paciente em MHC-PMS.

17.8 Usando exemplos, ilustre os diferentes tipos de adaptadores necessários para oferecer suporte à composição sequencial, à composição hierárquica e à composição aditiva.

17.9 Projete as interfaces de componentes que podem ser usadas em um sistema para uma sala de controle de emergência. Você deve projetar interfaces para um componente de registro de chamadas que registre as chamadas realizadas e um componente de descoberta do veículo que, dado um código postal (CEP) e um incidente, encontre o veículo adequado mais próximo a ser despachado para o incidente.

17.10 Foi sugerido que uma autoridade independente de certificação deveria ser estabelecida. Fornecedores submeteriam seus componentes a essa autoridade, que validaria se o componente era confiável. Quais seriam as vantagens e as desvantagens de tal autoridade de certificação?

REFERÊNCIAS

BOEHM, B. W.; ABTS, C.; BROWN, A. W.; CHULANI, S.; CLARK, B. K.; HOROWITZ, E. et al. *Software Cost Estimation with COCOMO II*. Upper Saddle River, NJ.: Prentice Hall, 2000.

COUNCILL, W. T.; HEINEMAN, G. T. Definition of a Software Component and its Elements. In: HEINEMAN, G. T.; COUNCILL, W. T. (Orgs). *Component-based Software Engineering*. Boston: Addison-Wesley, 2001, p. 5-20.

JACOBSON, I.; GRISS, M.; JONSSON, P. Software Reuse. Reading, Mass.: Addison-Wesley, 1997.

KOTONYA, G. The CBSE Process: Issues and Future Visions. *Proc. 2a CBSEnet workshop*, Budapeste, Hungria, 2003.

LAU, K.-K.; WANG, Z. Software Component Models. *IEEE Trans. on Software Eng.*, v. 33, n. 10, 2007, p. 709-724.

MEYER, B. Design by Contract. *IEEE Computer*, v. 25, n. 10, 1992, p. 40-51.

MEYER, B. The Grand Challenge of Trusted Components. *ICSE 25*: *Int. Conf. on Software Eng*, Portland, Oregon: IEEE Press, 2003.

MILI, H.; MILI, A.; YACOUB, S.; ADDY, E. *Reuse-based Software Engineering*. Nova York: John Wiley & Sons, 2002.

POPE, A. *The CORBA Reference Guide*: Understanding the Common Object Request Broker Architecture. Harlow, Reino Unido: Addison-Wesley, 1997.

SZYPERSKI, C. *Component Software*: Beyond Object-oriented Programming, 2a ed. Harlow, Reino Unido: Addison-Wesley, 2002.

WARMER, J.; KLEPPE, A. *The Object Constraint Language*: Getting your models ready for MDA. Boston: Addison-Wesley, 2003.

WEINREICH, R.; SAMETINGER, J. Component Models and Component Services: Concepts and Principles. In: HEINEMAN, G. T.; COUNCILL, W. T. (Orgs). *Component-Based Software Engineering*. Boston: Addison-Wesley, 2001, p. 33-48.